

IHC - Practical Approach to Data Mining and Analytics

Modular Assignment No. 4

Module 4: Data Analytics Using Programming Tools | Duration: 4 hours

Problem Statement

Design and implement a comprehensive Data Analytics solution that demonstrates mastery of programming tools, visualization frameworks, and data structures for data analysis. Students will build an end-to-end analytics platform using multiple programming languages and visualization tools while optimising data structures for efficient analysis of large-scale datasets.

Dataset Sources

Students must work with **at least 4 datasets** of different types and sizes to demonstrate tool versatility:

Structured Data Sources

- **Relational Datasets:**
 - **Northwind Database** (Classic SQL dataset)
 - **Sakila DVD Rental** (MySQL sample database)
 - **World Bank Development Indicators** (CSV/API format)
 - **Financial Markets Data** (Yahoo Finance, Alpha Vantage API)
 - **Government Statistics** (Census Bureau, Bureau of Labor Statistics)

Semi-Structured Data Sources

- **JSON/XML Data:**
 - **Twitter API** (Social media analytics)
 - **Reddit API** (Comment analysis)
 - **News API** (Article analysis)
 - **Weather API** (Time series data)
 - **GitHub API** (Repository analytics)
- **NoSQL Databases:**
 - **MongoDB Atlas** (Document-based datasets)
 - **Firestore** (Real-time data)
 - **Elasticsearch** (Search analytics)

Unstructured Data Sources

- **Text Data:**
 - **Project Gutenberg** (Literature analysis)
 - **arXiv Papers** (Academic text mining)
 - **Product Reviews** (Amazon, Yelp datasets)
 - **News Articles** (Reuters, BBC datasets)
- **Image/Multimedia:**
 - **CIFAR-10/100** (Image classification datasets)
 - **Kaggle Image Datasets** (Various computer vision tasks)
 - **Audio Files** (Speech recognition, music analysis)

Big Data Sources

- **Large-Scale Datasets (>1GB):**
 - **NYC Taxi Trip Records** (Transportation analytics)
 - **Wikipedia Dumps** (Text processing)
 - **Stack Overflow Data** (Developer analytics)
 - **Reddit Comments** (Social media analysis)
 - **Google Books Ngrams** (Language analysis)

Real-time/Streaming Data

- **APIs for Live Data:**
 - **Stock Market APIs** (Real-time trading data)
 - **IoT Sensor APIs** (Environmental monitoring)
 - **Social Media Streams** (Twitter, Facebook Graph API)
 - **Web Analytics** (Google Analytics API)

What is Expected

Students must demonstrate comprehensive understanding of Data Analytics programming tools through practical implementation of multi-language analytics platforms, advanced visualization systems, and optimised data structures. The solution should showcase tool integration, performance optimization, and scalable analytics architecture.

Rubrics and Evaluation Criteria

1. Programming Language Proficiency (25 points)

- **Excellent (23-25):** Mastery of multiple languages with advanced features and optimization
- **Good (18-22):** Good proficiency with appropriate language selection for tasks
- **Satisfactory (13-17):** Basic proficiency with some language limitations
- **Needs Improvement (0-12):** Inadequate programming skills or poor language utilization

Assessment Components

- **Python Mastery:** Advanced pandas, NumPy, scikit-learn, matplotlib, seaborn usage
- **R Proficiency:** Data manipulation with dplyr, visualization with ggplot2, statistical analysis
- **SQL Excellence:** Complex queries, window functions, CTEs, stored procedures
- **Additional Languages:** JavaScript (D3.js), Scala (Spark), Julia (high-performance computing)
- **Code Quality:** Clean, efficient, well-documented code with proper error handling
- **Performance Optimization:** Memory management, vectorization, parallel processing

2. Visualization Tools and Frameworks (25 points)

- **Excellent (23-25):** Multiple advanced visualization tools with interactive dashboards
- **Good (18-22):** Good visualization implementation with appropriate tool selection
- **Satisfactory (13-17):** Basic visualizations with standard tools
- **Needs Improvement (0-12):** Inadequate or poor visualization implementation

Assessment Components

- **Static Visualizations:** matplotlib, seaborn, ggplot2 with publication-quality output
- **Interactive Dashboards:** Plotly Dash, Streamlit, Shiny applications
- **Business Intelligence:** Tableau, Power BI integration and advanced features
- **Web-based Visualization:** D3.js, Observable, custom JavaScript implementations
- **Specialized Tools:** Gephi (networks), deck.gl (geospatial), Three.js (3D)
- **Dashboard Design:** User experience, responsive design, accessibility compliance

3. Data Structures and Optimization (25 points)

- **Excellent (23-25):** Advanced data structures with performance optimization for Big Data
- **Good (18-22):** Good data structure selection with appropriate optimization
- **Satisfactory (13-17):** Basic data structures with some optimization
- **Needs Improvement (0-12):** Inadequate data structure knowledge or poor optimization

Assessment Components

- **Pandas Optimization:** Efficient DataFrames, categorical data, memory optimization
- **NumPy Arrays:** Vectorization, broadcasting, advanced indexing

- **Database Design:** Normalization, indexing, query optimization
- **Big Data Structures:** Spark DataFrames, Dask arrays, distributed computing
- **Memory Management:** Chunk processing, lazy evaluation, garbage collection
- **Algorithm Complexity:** Time and space complexity analysis and optimization

4. Package Ecosystem and Integration (25 points)

- **Excellent (23-25):** Extensive package integration with custom solutions and APIs
- **Good (18-22):** Good package utilization with proper integration
- **Satisfactory (13-17):** Basic package usage with some integration
- **Needs Improvement (0-12):** Limited package knowledge or poor integration

Assessment Components

- **Data Manipulation:** pandas, dplyr, data.table, polars
- **Machine Learning:** scikit-learn, caret, tidymodels, MLlib
- **Visualization:** Complete ecosystem integration (matplotlib + seaborn + plotly)
- **Statistical Analysis:** scipy, statsmodels, R statistical packages
- **Web Development:** Flask/Django, Shiny, Node.js for analytics applications
- **API Integration:** RESTful APIs, GraphQL, WebSocket connections

Deliverables

1. Multi-Language Analytics Platform (40%)

- **Polyglot Implementation:** Analytics pipeline using Python, R, SQL, and one additional language
- **API Development:** RESTful analytics API with proper documentation
- **Database Integration:** Multi-database connectivity (SQL, NoSQL, graph databases)
- **Performance Benchmarks:** Comparative analysis of language/tool performance

2. Interactive Visualization Suite (35%)

- **Dashboard Portfolio:** Multiple dashboards using different tools (Tableau, Plotly, D3.js)
- **Responsive Design:** Mobile-friendly, accessible visualizations
- **Real-time Updates:** Live data streaming and dynamic visualization updates
- **Custom Components:** Original visualization components and widgets

3. Technical Documentation and Best Practices (25%)

- **Architecture Guide:** System design and component interaction documentation
- **Performance Analysis:** Benchmarking results and optimization recommendations
- **Code Repository:** Well-organised, documented codebase with CI/CD pipeline
- **User Manual:** Comprehensive guide for platform usage and maintenance

Technical Requirements

Core Programming Stack

- **Python 3.9+:** pandas, NumPy, matplotlib, seaborn, plotly, streamlit, flask
- **R 4.0+:** tidyverse, ggplot2, shiny, data.table, caret
- **SQL:** PostgreSQL/MySQL with advanced query features
- **JavaScript:** D3.js, Observable, Node.js for web integration

Visualization Requirements

- **Business Intelligence:** Tableau Public or Power BI integration
- **Interactive Dashboards:** Plotly Dash, Streamlit, or Shiny applications
- **Custom Visualizations:** D3.js or custom JavaScript implementations
- **Mobile Responsive:** Progressive web app capabilities

Data Structure Optimization

- **Memory Efficiency:** Handle datasets larger than available RAM
- **Processing Speed:** Vectorized operations, parallel processing
- **Scalability:** Distributed computing with Spark or Dask
- **Real-time Processing:** Streaming Data Analysis capabilities

Advanced Options (Bonus Points)

- **Cloud Integration:** AWS/GCP/Azure analytics services
- **Containerization:** Docker deployment with orchestration
- **Machine Learning Ops:** MLflow, Kubeflow integration
- **Graph Analytics:** Neo4j, NetworkX for network analysis
- **GPU Acceleration:** CUDA, Rapids for high-performance computing

Timeline and Milestones

Hour 1: Multi-Language Data Ingestion and Processing

- Environment setup and package installation
- Data ingestion from multiple sources using different languages
- Basic data exploration and quality assessment
- Cross-language data exchange and format conversion

Hour 2: Advanced Data Structures and Optimization

- Efficient data structure implementation
- Memory optimization and performance tuning
- Parallel processing and vectorization
- Database design and query optimization

Hour 3: Visualization and Dashboard Development

- Static visualization creation with multiple tools
- Interactive dashboard development
- Custom visualization component creation
- Responsive design implementation

Hour 4: Integration and Deployment

- API development and integration testing
- Performance benchmarking and optimization
- Documentation and user guide creation
- Deployment and accessibility testing

Specific Implementation Requirements

Data Processing Pipeline

```
# Example structure - students must implement
class AnalyticsPlatform:
    def __init__(self, config):
        self.python_engine = PythonAnalytics()
        self.r_engine = RAnalytics()
        self.sql_engine = SQLAnalytics()
        self.viz_engine = VizualizationEngine()

    def ingest_data(self, sources, formats):
        # Multi-format data ingestion
        pass

    def process_data(self, operations, language='auto'):
        # Language-optimized processing
        pass

    def create_visualization(self, data, chart_type, tool='auto'):
        # Multi-tool visualization
        pass

    def deploy_dashboard(self, components, framework):
        # Dashboard deployment
        pass
```

Visualization Framework

```
# Example structure - students must implement
class VizFramework:
    def __init__(self):
        self.static_engines = ['matplotlib', 'seaborn', 'ggplot2']
        self.interactive_engines = ['plotly', 'bokeh', 'd3js']
        self.dashboard_engines = ['streamlit', 'dash', 'shiny']

    def create_chart(self, data, chart_type, engine, config):
        # Universal chart creation
        pass

    def build_dashboard(self, components, layout, responsive=True):
        # Dashboard builder
        pass

    def export_visualization(self, chart, format, quality='high'):
        # Multi-format export
        pass
```

Data Structure Optimizer

```
# Example structure - students must implement
class DataOptimizer:
    def __init__(self):
        self.memory_threshold = self.get_available_memory()
        self.cpu_cores = self.get_cpu_count()

    def optimize_dataframe(self, df, operations):
        # Memory and speed optimization
        pass

    def chunk_processor(self, data_source, chunk_size='auto'):
        # Large dataset processing
        pass

    def parallel_process(self, data, function, n_jobs=-1):
        # Parallel processing optimization
        pass

    def benchmark_performance(self, operations, data_sizes):
        # Performance benchmarking
        pass
```

Evaluation Scenarios

Scenario 1: Financial Analytics Platform

- **Data Sources:** Stock prices, economic indicators, news sentiment, social media
- **Languages:** Python (ML), R (statistics), SQL (data warehouse), JavaScript (frontend)
- **Visualizations:** Real-time trading dashboard, risk assessment charts, portfolio analytics
- **Structures:** Time-series optimization, financial calculations, real-time data streams
- **Expected Output:** Complete financial analytics platform with live dashboard

Scenario 2: Healthcare Data Analytics

- **Data Sources:** Patient records, medical imaging, clinical trials, genomic data
- **Languages:** Python (image processing), R (biostatistics), SQL (HIPAA compliance)
- **Visualizations:** Patient dashboards, clinical trial results, epidemiological maps
- **Structures:** Medical data standards (HL7, FHIR), privacy-preserving analytics
- **Expected Output:** Healthcare analytics platform with regulatory compliance

Scenario 3: E-Commerce Intelligence

- **Data Sources:** Transaction data, customer behaviour, inventory, market trends
- **Languages:** Python (recommendation systems), R (customer analytics), SQL (data warehouse)
- **Visualizations:** Sales dashboards, customer journey maps, inventory optimization
- **Structures:** Graph databases for recommendations, real-time analytics
- **Expected Output:** E-Commerce intelligence platform with predictive analytics

Scenario 4: Smart City Analytics

- **Data Sources:** IoT sensors, traffic data, energy consumption, citizen feedback
- **Languages:** Python (IoT processing), R (urban planning), SQL (infrastructure data)
- **Visualizations:** City dashboards, traffic flow maps, energy optimization charts
- **Structures:** Time-series for sensor data, geospatial analytics, real-time processing
- **Expected Output:** Smart city analytics platform with real-time monitoring

Performance Benchmarks

Processing Performance

- **Data Ingestion:** >1GB/minute for structured data
- **Data Processing:** >100K records/second for standard operations
- **Visualization Rendering:** <2 seconds for complex charts
- **Dashboard Loading:** <5 seconds for complete dashboard
- **Memory Efficiency:** Process datasets 5x larger than available RAM

Scalability Metrics

- **Concurrent Users:** Support 100+ simultaneous dashboard users
- **Data Volume:** Handle datasets up to 10GB efficiently
- **Real-time Processing:** <1 second latency for streaming data
- **API Response:** <200ms for standard analytics queries

Code Quality Standards

- **Test Coverage:** >80% unit test coverage
 - **Documentation:** Complete API documentation with examples
 - **Performance:** $O(n \log n)$ or better for core algorithms
 - **Maintainability:** Modular design with clear separation of concerns
-

Innovation Challenges (Bonus)

1. Novel Visualization Techniques (15 bonus points)

- Develop new visualization methods for complex data relationships
- Create augmented reality or virtual reality data visualization
- Implement natural language interfaces for data exploration
- Design accessibility-first visualization components

2. Cross-Platform Integration (15 bonus points)

- Seamless integration between Python, R, and JavaScript environments
- Real-time data synchronization across multiple tools
- Universal data format converter with metadata preservation
- Cross-language debugging and profiling tools

3. AI-Powered Analytics Assistant (20 bonus points)

- Natural language query interface for Data Analytics
- Automated insight generation and anomaly detection
- Intelligent chart recommendation based on data characteristics
- Auto-generated narrative explanations for analytical results

4. Performance Innovation (10 bonus points)

- GPU-accelerated analytics for massive datasets
 - Quantum computing integration for specific analytical tasks
 - Edge computing deployment for IoT analytics
 - Blockchain integration for data provenance and security
-

Advanced Topics (Optional)

1. Real-Time Analytics Architecture

- Apache Kafka for data streaming
- Redis for caching and real-time storage
- WebSocket connections for live dashboard updates
- Event-driven architecture patterns

2. Cloud-Native Analytics

- Serverless analytics functions (AWS Lambda, Google Cloud Functions)
- Container orchestration (Kubernetes) for analytics workloads
- Cloud data warehouses (BigQuery, Redshift, Snowflake)
- Multi-cloud deployment strategies

3. Advanced Data Structures

- Column-oriented storage for analytics (Parquet, ORC)
 - Graph databases for relationship analytics (Neo4j, ArangoDB)
 - Vector databases for similarity search (Pinecone, Weaviate)
 - Time-series databases (InfluxDB, TimescaleDB)
-

Industry Integration Requirements

1. Business Intelligence Standards

- **Tableau Integration:** Advanced calculations, custom visualizations, server deployment
- **Power BI Integration:** Custom visuals, DAX functions, Power Query optimization
- **Excel Integration:** Advanced formulas, Power Pivot, VBA automation
- **Google Analytics:** Custom dimensions, reporting API, data studio integration

2. Statistical Software Integration

- **SPSS Integration:** Syntax automation, custom procedures
- **SAS Integration:** Macro programming, enterprise deployment
- **Stata Integration:** Custom commands, statistical analysis
- **MATLAB Integration:** Toolbox development, algorithm optimization

3. Enterprise Tool Integration

- **Jupyter Integration:** Custom kernels, enterprise deployment, collaborative features
- **Apache Spark:** Scala/Python integration, cluster management, ML pipelines

- **Databricks:** Collaborative notebooks, Delta Lake, MLflow integration
 - **Snowflake:** Data sharing, secure views, JavaScript UDFs
-

Submission Requirements

1. Complete Analytics Platform

- **Source Code:** Multi-language implementation with proper structure
- **API Documentation:** RESTful API with OpenAPI specification
- **Deployment Guide:** Docker containerization and deployment instructions
- **Performance Benchmarks:** Comprehensive performance analysis and optimization

2. Visualization Portfolio

- **Dashboard Collection:** Multiple dashboards using different tools and frameworks
- **Interactive Components:** Custom visualization widgets and components
- **Responsive Design:** Mobile-friendly and accessible implementations
- **User Experience:** Navigation, filtering, and interaction design

3. Technical Excellence Documentation

- **Architecture Guide:** System design, data flow, and component interactions
 - **Best Practices:** Code standards, optimization techniques, security considerations
 - **Troubleshooting Guide:** Common issues, debugging techniques, performance tuning
 - **Future Roadmap:** Enhancement suggestions and scalability planning
-

Academic Integrity and Professional Standards

Code Quality and Documentation

- Industry-standard coding practices and conventions
- Comprehensive inline documentation and README files
- Version control with meaningful commit messages
- Automated testing and continuous integration

Data Security and Privacy

- Proper handling of sensitive data with encryption
- GDPR and privacy compliance considerations
- Secure API development with authentication and authorization
- Data anonymization and pseudonymization techniques

Performance and Scalability

- Efficient algorithm implementation with complexity analysis
 - Memory optimization and resource management
 - Horizontal and vertical scaling strategies
 - Load testing and performance monitoring
-

Support Resources

Technical Documentation

- Programming language reference guides and best practices
- Visualization tool tutorials and advanced techniques
- Database optimization and query performance guides
- API development and integration standards

Development Environment

- Pre-configured development environments (Docker containers)
- Cloud-based development platforms (GitHub Codespaces, Google Colab)
- Package management and dependency resolution guides
- Debugging tools and performance profiling utilities

Industry Standards

- Data visualization best practices and design principles
- Analytics architecture patterns and design guidelines
- Performance benchmarking methodologies
- Code review checklists and quality assurance processes

Total Points: 100

Bonus Points: Up to 60

Passing Grade: 70 points

Time Limit: 4 hours